

COURS DE GENIE LOGICIEL



ISIG - GOMA

INSTITUT SUPERIEUR
D'INFORMATIQUE ET DE
GESTION
BP : 841 GOMA

E-mail:
info@isig.ac.cd

REPUBLIQUE DEMOCRATIQUE DU CONGO
ENSEIGNEMENT SUPERIEUR ET UNIVERSITAIRE

E.S.U

*EXAMEN DU COURS DE GENIE LOGICIEL :
CONCEPTION D'UNE APPLICATION DE
GESTION DU TRAFIC ROUTIER DU QUARTIER
LES VOLCANS EN VILLE DE GOMA*

Effectué par : GROUPE 6

Superviseur : ---

Promotion : LIC3 & LIC3

Option :

- Licence en Informatique appliquée à la gestion d'une entreprise
- Licence en système informatique

Année Académique

2025- 2026

MEMBRES DU GROUPE

1. KAMARA DIDI HONORA
2. KAMANYA KAPITENI DANNY
3. KALENGA KAMONA AKEEM
4. KAKULE NDUNGO ELOQUENT
5. KAKULE MITANDI SAMUEL
6. KAHINDO BAENI SARAH
7. KAHINDO MPANZI SOLANGE
8. KAGHENI KABUYA BONFORT
9. KAFINDO MUSEKA HYANCINTHE
10. KAFFEKE MBALE CHRISTIAN

1. INTRODUCTION ET CONTEXTE DE L'ÉTUDE

Dans le cadre de notre évaluation académique en Génie Logiciel, il nous a été confié la tâche de concevoir et de développer une solution logicielle innovante répondant à une problématique locale majeure : la gestion du trafic routier dans le quartier "Les Volcans" de la ville de Goma.

La croissance démographique et l'augmentation du parc automobile dans cette zone engendrent des défis quotidiens pour la mobilité urbaine (embouteillages, accidents, difficultés d'intervention). Notre objectif, à travers ce projet nommé **VolcanWay**, a été d'apporter une approche d'ingénierie logicielle rigoureuse pour modéliser, structurer et déployer un système d'information capable de centraliser les données de trafic, de signaler les incidents en temps réel et de fournir des outils d'aide à la décision pour les conducteurs et les autorités locales.

2. DEMARCHE METHODOLOGIQUE ET CYCLE DE VIE

Pour mener à bien ce projet de bout en bout, nous avons adopté une démarche méthodique couvrant l'intégralité du cycle de vie du développement logiciel, de la compréhension abstraite jusqu'au déploiement technique.

2.1. Compréhension du Sujet et Analyse des Besoins

La première étape de notre travail a consisté à cerner précisément le périmètre du problème. Nous avons identifié les différents acteurs impliqués et recueilli leurs besoins respectifs :

- **Les conducteurs et les piétons (qui sont dans le système comme des utilisateurs standard)**, qui ont besoin d'être informés en temps réel de l'état des routes pour optimiser leurs trajets.
- **La police de circulation et les administrateurs**, qui nécessitent un tableau de bord (Dashboard) pour réguler le trafic, valider les signalements sur le terrain et analyser les statistiques de la voirie.

2.2. Modélisation du Système (Approche UML)

Un dossier annexe accompagne ce livrable et contient nos différents diagrammes :

- **Diagramme de Cas d'Utilisation** : Pour illustrer les interactions de haut niveau entre les utilisateurs (citoyens, agents, administrateurs) et les fonctionnalités du système (signalement d'incidents, consultation de trafic routier, gestion des utilisateurs).
- **Diagrammes de Séquence et d'Activité** : Pour détailler la logique temporelle et la chronologie des processus critiques, notamment le flux allant du signalement d'un accident jusqu'à sa validation par un agent de police.
- **Diagramme de Classes** : Pour structurer le cœur de notre application et notre base de données relationnelle, mettant en évidence les entités fondamentales (Utilisateurs, Routes, Incidents, Alertes) et les relations qui les unissent.
- **Diagramme de Déploiement** : Pour cartographier l'architecture physique et logique de notre solution web, répartie entre le client (navigateur web/mobile), le serveur d'application et le serveur de base de données.

2.3. Implémentation Technologique et justification

Sur la base de ces modèles solides, nous sommes passés à la phase de codage en adoptant une architecture moderne séparée (Frontend/Backend) afin de garantir la scalabilité et la maintenabilité du code. Le choix de chaque outil a été fait en fonction de sa performance, sa simplicité d'utilisation et son adéquation avec les objectifs du projet.

- **Base de données (MySQL)** : Conception d'un schéma relationnel optimisé garantissant l'intégrité et la persistance sécurisée des données. Stocke toutes les données du système :

- Utilisateurs
- Incidents routiers
- Alertes
- Assure la persistance et la cohérence des données

- **Backend (Python / FastAPI)** : Développement d'une API RESTful robuste chargée de gérer la logique métier complexe, la sécurité (authentification par token) et la communication asynchrone avec la base de données.

FastAPI a été utilisé pour le backend afin de gérer la logique métier et les communications entre le frontend et la base de données. Il a été un choix pour sa simplicité, sa rapidité et sa robustesse dans la création d'API REST. Les raisons principales sont :

- Performance élevée : FastAPI est l'un des frameworks Python les plus rapides pour créer des API.
- Développement rapide : il permet de créer des endpoints très rapidement avec peu de code.
- Validation automatique des données : grâce à Pydantic, les données envoyées par le frontend sont automatiquement vérifiées.
- Documentation automatique : FastAPI génère automatiquement une documentation interactive (Swagger UI), ce qui facilite les tests.
- Compatibilité avec Python : étant basé sur Python, il s'intègre facilement avec d'autres bibliothèques.

- **Frontend (React / TypeScript)** : Création d'une interface utilisateur web interactive, pensée avec une approche "Mobile-First" pour être utilisable par les conducteurs sur leurs smartphones, intégrant une cartographie et des visualisations de données.

React a été pour nous un choix pour le développement du frontend car il s'agit d'une bibliothèque moderne et performante permettant de créer des interfaces utilisateur dynamiques. Les principales raisons de ce choix sont :

- Architecture basée sur les composants : React permet de découper l'interface en composants réutilisables (formulaires, tableaux, dashboards), ce qui rend le code plus organisé et maintenable.
- Mise à jour rapide de l'interface (Virtual DOM) : React met à jour uniquement les parties nécessaires de la page, ce qui améliore fortement les performances.
- Expérience utilisateur fluide : l'application devient plus réactive sans rechargement complet des pages.

- Large communauté et support : React est très utilisé dans l'industrie, ce qui facilite l'apprentissage et la résolution de problèmes.

Avec la combinaison de **Vite** pour son démarrage rapide du serveur de développement, recharge instantanée (Hot Reload) lors de modifications du code. Elle réduit le temps d'attente pendant le développement.

Combiné aussi avec TypeScript qui est une extension de JavaScript qui ajoute le typage statique. Parmi ces avantages, il y a détection des erreurs avant exécution, meilleure lisibilité et compréhension du code, réduction des bugs dans les applications complexes.

Le choix de React nous a permis donc de construire une interface moderne, rapide et évolutive pour les utilisateurs du système.

2.4. Phase de Tests

Avant de considérer l'application comme exploitable, nous avons effectué des séries de tests rigoureux en environnement local. Ces tests nous ont permis de valider la connectivité de bout en bout (du clic de l'utilisateur jusqu'à l'enregistrement en base de données), d'assurer la correcte remontée des alertes de trafic, et de vérifier le système de permissions selon les rôles des utilisateurs.

2.5 La collaboration entre membres de l'équipe

Nous avons adopté une approche de développement collaboratif en utilisant la plateforme GitHub. Cet outil nous a permis de centraliser le code source, de suivre les modifications et de faciliter la collaboration entre les membres de l'équipe.

3. PERSPECTIVES

Bien que notre application de gestion de trafic routier réponde aux besoins essentiels définis dans le cadre de ce projet, plusieurs améliorations et fonctionnalités supplémentaires pourraient être envisagées à long terme afin d'enrichir le système et d'augmenter son efficacité.

Parmi les perspectives d'évolution, nous pouvons citer l'intégration de systèmes intelligents basés sur l'analyse des données, permettant de prédire les embouteillages et d'optimiser la circulation en temps réel. L'ajout de capteurs routiers et de caméras de surveillance pourrait également permettre une collecte automatique et continue des informations liées au trafic.

De plus, le développement d'une application mobile destinée aux usagers offrirait la possibilité de recevoir des alertes en temps réel concernant les incidents, les ralentissements ou les déviations. Une autre amélioration importante serait l'intégration d'un système de géolocalisation pour suivre les flux de circulation et proposer des itinéraires alternatifs.

Enfin, l'intégration de technologies d'intelligence artificielle pourrait contribuer à automatiser certaines prises de décision, comme la régulation des feux de signalisation ou la gestion des priorités en cas d'urgence.

Ces différentes perspectives montrent que ce projet peut évoluer vers une solution plus complète, intelligente et adaptée aux réalités du trafic urbain moderne.

4. CONCLUSION

Ce projet de conception dédié au quartier "Les Volcans" a été une excellente opportunité pour notre groupe de mettre en pratique et d'articuler l'ensemble des concepts théoriques du génie logiciel vus en cours. En suivant les phases d'analyse, de modélisation UML et d'implémentation, nous avons abouti à une réalisation expérimentale, fonctionnelle et évolutive. Nous sommes convaincus que les fondations logicielles posées par "VolcanWay" démontrent la faisabilité technique d'une gestion intelligente du trafic et pourraient, à terme, être étendues à l'échelle de toute la ville de Goma.